

PROVA G1 – TIPO 1

Gabarito das objetivas

Questão Resp.

01	B
02	B
03	B
04	C
05	E
06	B
07	E
08	C
09	A
10	C
11	C
12	A
13	C
14	B
15	C

1 – B.

I, II e III estão corretas. A primeira abordagem descreve comandos e alteração de estado, característica imperativa. A segunda privilegia a descrição do resultado, característica declarativa. A IV é falsa: linguagens declarativas não necessariamente eliminam todo controle sobre a execução.

2 – B.

O uso de funções como valores e `map()`/`filter()` está associado ao paradigma funcional. A imutabilidade e a redução de efeitos colaterais são características importantes desse paradigma.

3 – B.

Atributos protegidos contra acesso direto → **encapsulamento**.

`Carro extends Veiculo` → **herança**.

Métodos com implementações diferentes → **polimorfismo**.

4 – C.

No paradigma lógico, o programador descreve **fatos e regras**, e o mecanismo de inferência encontra respostas para as consultas.

5 – E.

Todas estão corretas. Paradigmas não são apenas classificações de linguagens; eles representam diferentes formas de estruturar o raciocínio para resolver problemas.



6 – B.

Evitar repetir a mesma validação → **DRY** (*Don't Repeat Yourself*).

Não implementar uma funcionalidade que ainda não é necessária → **YAGNI** (*You Aren't Gonna Need It*).

7 – E.

A primeira classe tem responsabilidades relacionadas → **alta coesão**.

A segunda mistura responsabilidades distintas → **baixa coesão**.

Alterações em uma classe exigindo alterações em outra indicam **alto acoplamento**.

Portanto, I, II, III e IV estão corretas.

8 – C.

Se cada nova operação exige modificar a classe existente, há dificuldade de extensão.

Isso contraria o **OCP — Open/Closed Principle**. Uma abstração permitiria adicionar novas operações sem alterar continuamente o código existente.

9 – A.

- várias responsabilidades → **SRP**
- subclasse não substitui adequadamente a base → **LSP**
- dependência de implementação concreta → **DIP**
- interface obriga métodos desnecessários → **ISP**

10 – C.

Passar `ServicoDePagamento` por meio do construtor reduz a dependência de `Checkout` em relação à implementação concreta e permite diferentes implementações. Isso caracteriza **DIP**.

Importante: a interface **não elimina a dependência**. `Checkout` continua dependendo de `ServicoDePagamento`; a diferença é que depende de uma **abstração**, e não de uma implementação concreta.

11 – C.

I, II e IV estão corretas. Microservices permitem escalabilidade independente, mas introduzem desafios adicionais de comunicação e operação.

12 – A.

O padrão não exige necessariamente um servidor físico exclusivo. O importante é o **isolamento lógico dos dados**, podendo inclusive existir compartilhamento da infraestrutura física.

13 – C.

Depois que uma migração foi aplicada, não se deve simplesmente modificá-la. Cria-se uma nova migração, por exemplo:

```
V3__adicionar_telefone_usuario.sql
```

14 – B.

I, II e III estão corretas. A IV é falsa porque centralizar configuração **não garante automaticamente atualização dinâmica** em todos os serviços.



15 – C.

I, II e IV estão corretas. Profiles permitem manter o mesmo código e selecionar conjuntos diferentes de configurações para cada ambiente.

Questão 16 – Tipo 1 — ISP

A questão apresenta uma classe `Robo` obrigada a implementar métodos da interface `Trabalhador` que não fazem sentido para ela, inclusive lançando exceções.

a) Resposta esperada

O princípio violado é o **ISP — Interface Segregation Principle (Princípio da Segregação de Interfaces)**.

A interface `Trabalhador` possui métodos relacionados a diferentes comportamentos. A classe `Robo` é obrigada a implementar métodos que não utiliza ou que não fazem sentido para seu contexto. Como consequência, alguns métodos precisam lançar `UnsupportedOperationException`.

Isso indica que a interface é muito abrangente e deveria ser dividida em interfaces menores e mais específicas.

b) Solução esperada

Por exemplo:

```
interface Trabalhavel {
    void trabalhar();
}

interface Programavel {
    void programar();
}

interface Gerenciavel {
    void gerenciarEquipe();
}

interface Alimentavel {
    void comer();
}
```

Então cada classe implementaria apenas aquilo que realmente faz sentido:

```
class Robo implements Trabalhavel, Programavel {

    public void trabalhar() {
        // ...
    }

    public void programar() {
```



```
        // ...  
    }  
}
```

Enquanto uma pessoa poderia implementar:

```
class Funcionario implements Trabalhavel, Alimentavel {  
  
    public void trabalhar() {  
        // ...  
    }  
  
    public void comer() {  
        // ...  
    }  
}
```

Explicação:

O problema não é a existência de interfaces, mas uma **interface grande demais**.

A ideia do ISP é:

Nenhuma classe deve ser obrigada a depender de métodos que não utiliza.

E aqui temos exatamente isso.

Nesta questão, **ISP é efetivamente a resposta esperada**. Diferentemente de um problema na hierarquia de herança, onde uma sub-classe não consegue cumprir o contrato de uma super classe, aqui o problema está explicitamente na interface que obriga o Robo a implementar comportamentos desnecessários.